BRAINSTER

Events (2)

Brainster Web Development Academy

Table of contents

Window events

- Similar to how we can “**listen**” for events on DOM elements, we can also listen for events on the window itself.
- The most notable and most commonly used among these are “**load**”, “**unload**” and “**beforeunload**”.
- Additional ones include: “**resize**”, “**scroll**”, “**hashchange**” etc.

Window events

```
function onLoad() {
    alert(“Welcome!”);
}

function onUnload(event) {
    // this is the only way this event works, alerts/messages are ignored.
    event.returnValue = 'Are you sure you want to leave this page?';
    return true;
}

window.addEventListener(‘load’, onLoad);
window.addEventListener(‘beforeunload’, onUnload);
```


Exercise I

1. In CSS, give the body height 10000vh.
2. Define a function onScroll and attach it on a ‘scroll’ event on the window.
3. Inside the ‘scroll’ event you have access to the window.scrollY property which gives you the exact position of the page you are at.
4. Using the window.scrollY check if you have scrolled:
   a) past 1000px on the page, and if you had, color the body red.
   b) otherwise, color the body green.

LocalStorage

The simplest way to store data when you close and reopen the page is localStorage. You can find localStorage data in the browser inspect element. There are 2 simple methods to store and get the data.

- **setItem()** - when passed a key - name and value, will add that key to the storage, or update that key's value if it already exists.
- **getItem()** - when passed a key - name, will return that key's value.

```
localStorage.setItem('nameOfTheValue', value);
let storedValue = localStorage.getItem('nameOfTheValue');
```


Exercise II

Continue in the same document from the last presentation.

1. Change the submit function: store the variables defined in the previous exercises (so they remain after a reload). **hint**: **localStorage.setItem()**

2. Create an onLoad function which checks if values in localStorage exist & loads them in console. **hint**: **localStorage.getItem();**

3. Add a “**load**” listener to the window, triggering the onLoadfunction, so it invokes on reload.

Exercise III

Continue on the last exercise.

1. Define a onUnload function, which triggers a “confirm” dialog asking the user whether they are sure they want to leave.

2. Add a “**beforeunload**” listener to the window, triggering the onUnload function.

3. Add up to the onLoad function so it re-populates the input & the textarea in html with the appropriate values from localStorage.

(make sure you are refreshing the page with ctrl + f5 so it doesn’t read the values from cache, but from localStorage)

Form events

The “change” event

- The “**change**” event fires when a value changes in a text input, textarea, checkboxes, radio buttons and select elements.
- One thing to note with it is that in the cases of text input and textarea, the event fires **after** the input / textarea is de-focused.
- It’s particularly useful for handling select elements and checkboxes, while there are arguably better ways to handle text inputs and textareas.

The “change” event

```
let mySelect = document.querySelector('select');

function onChange() {
    console.log(mySelect.value);
}

mySelect.addEventListener('change', onChange);
```


Exercise IV

1. Add another div in html with a class of “row”.
2. Inside the div, add a label with the text “Profession”.
3. Next to the label, add a select box, with some profession options like “student”, “developer”, “designer”...
4. Define a profession variable, with no value.
5. Define a function called “onProfessionChange”, which takes the value of the select box from the document and assigns it to the profession variable.
6. Add a “change” event listener to the select box from the document, and invoke onProfessionChange when it happens.

The “input” event

- The "**input**" event is triggered immediately after the value of an **<input>**, **<textarea>**, or **<select>** element has been changed. This is in contrast to the "change" event, which waits for the focus to leave the element (in the case of text inputs and textareas).

- It fires instantly when the value changes, making it ideal for real-time feedback.


```
let mySelect = document.querySelector(‘input’);

function onInputChange() {
   console.log(mySelect.value);
}

mySelect.addEventListener(‘input’, onInputChange);
```

“focus” and “blur”

- “**focus**” and “**blur**” simply tell us whether an element is focused or not. This makes the most sense for interactive elements (input, select, button ...)
- It’s mostly useful for checking whether we have a valid input somewhere, whether we left a field empty before submission etc.
- It can also be useful for adding custom styles to focused elements.

“focus” and “blur”

```
let myInput = document.querySelector(‘input’);

function onFocus(event) {
   console.log(event.currentTarget, ‘focused’);
}

function onBlur(event) {
   console.log(event.currentTarget, ‘blurred’);
}

myInput.addEventListener(‘focus’, onFocus);
myInput.addEventListener(‘blur’, onBlur);
```


Exercise V

1. **Define two new function onBlur & onFocus and attach the two functions on ‘blur’ and ‘focus’ events on the input.**

2. **Inside the onFocus function:**
   a) Add a background-color of lightgrey on the element.

3. **Inside the onBlur function:**
   a) Remove the background-color of the element (make it white).
   b) Check if the element is empty, and if it is, give it a red border color to signal that it misses some data.
   c) Check if the element has some value, and if it has, give it a green border color to signal that it is complete.